

DISPENSA DELLA LEZIONE

Array, metodi e sintassi utile

Array, indici, metodi base, destructuring e spread operator.

CAPITOLO 1

Array

NOTA

Un array è una lista ordinata di valori racchiusa tra parentesi quadre. Si usa quando hai più elementi dello stesso argomento e vuoi tenerli in un unico contenitore.

Il nome dell'array va al plurale perché rappresenta un gruppo di valori, non uno solo.

Senza gli array, per conservare dieci nomi dovresti scrivere dieci variabili separate. Con un array li metti tutti insieme in una sola struttura, nello stesso ordine in cui li hai inseriti. Questo semplifica il codice e rende più facile lavorare sui dati in modo sistematico.

Un array è detto **omogeneo** quando tutti i suoi valori sono dello stesso tipo, ad esempio solo stringhe o solo numeri. È detto **eterogeneo** quando contiene tipi diversi, come stringhe, numeri e booleani insieme. Gli array omogenei sono più comuni nella pratica, perché è più raro avere dati di natura diversa nello stesso insieme.

Esempio

```
// Array omogeneo di stringhe
const studenti = ['Anna', 'Luca', 'Sara'];

// Array omogeneo di numeri
const voti = [8, 7, 9];

// Array eterogeneo: tipi diversi nello stesso array
const scheda = ['Marco', 18, true];
```

CAPITOLO 2

Indice, lunghezza e ciclo for

NOTA

L'indice è la posizione di un elemento nell'array. Si parte sempre da `0`, non da `1`.

`.length` restituisce il numero totale degli elementi. `console.table()` mostra l'array in forma tabellare nella console.

Per leggere un elemento devi scrivere il nome dell'array seguito dall'indice tra parentesi quadre: `studenti[0]` restituisce il primo elemento. Poiché l'indice parte da `0`, l'ultimo elemento si trova all'indice `length - 1`.

Il ciclo `for` e gli array si abbinano in modo naturale: la variabile del ciclo parte da `0`, sale di uno a ogni iterazione e si ferma quando raggiunge `.length`. In questo modo puoi leggere o elaborare ogni elemento della lista in modo automatico, senza ripetere il codice.

Esempio

```
const studenti = ['Anna', 'Luca', 'Sara', 'Marco'];

// Leggo il primo e l'ultimo elemento con l'indice
console.log(studenti[0]);           // 'Anna'
console.log(studenti[studenti.length - 1]); // 'Marco'

// Visualizzo l'array come tabella nella console
console.table(studenti);

// Scorro tutti gli elementi con un ciclo for
for (let i = 0; i < studenti.length; i++) {
  console.log(studenti[i]);
}
```

CAPITOLO 3

Metodi degli array

NOTA

I metodi sono azioni già pronte che puoi applicare direttamente su un array. Alcuni modificano l'array originale, altri restituiscono un nuovo array o un valore senza toccare l'originale.

Un array dichiarato con `const` può comunque essere modificato con i metodi: `const` blocca il riferimento al contenitore, non i suoi contenuti.

Gli array mettono a disposizione una serie di metodi già pronti per le operazioni più comuni: aggiungere o rimuovere elementi, cercare un valore, ordinare, filtrare o trasformare la lista. Conoscerli evita di scrivere codice lungo e ripetitivo per fare cose che JavaScript sa già fare in una riga. L'unica distinzione importante da tenere a mente è se un metodo modifica l'array originale oppure restituisce un nuovo array senza toccarlo: saperlo in anticipo previene errori difficili da individuare.

Lista dei metodi

Codice JavaScript	Risultato / Output	Note
<code>😊😊😊😊.push(😎)</code>	😊😊😊😊😎	Aggiunge in fondo
<code>😊😊😊😊.unshift(😎)</code>	😎😊😊😊😊	Aggiunge in testa
<code>😊😊😊😊.pop()</code>	😊😊😊😊	Rimuove l'ultimo elemento
<code>😊😊😬😎.shift()</code>	😊😊😎	Rimuove il primo elemento
<code>😊😊😬😎.reverse()</code>	😎😬😊😊	Inverte l'ordine degli elementi
<code>😊😊😬😎.includes(😬)</code>	true	Controlla se l'elemento esiste
<code>😊😊😬😎.at(2)</code>	😬	Prende l'elemento alla posizione 2
<code>😊😊😬😎.slice(0, 1)</code>	😊😊	Crea un nuovo array (estrazione)
<code>😊😊😬😎.splice(0, 1)</code>	😎😬😎	Modifica l'array eliminando elementi
<code>😊😊😬😎.splice(1, 1, ❤️, 💩)</code>	😊😊❤️💩😬😎	Rimuove e inserisce nuovi elementi
<code>😊😊😬😎.fill(😬)</code>	😬😬😬😬	Sostituisce tutti gli elementi
<code>😊😊😬😎.indexOf(😬)</code>	2	Trova l'indice della prima occorrenza
<code>😊😊😬😎.join('-')</code>	😊😊 - 😬 - 😬 - 😎	Unisce gli elementi in una stringa

Esempio

```
const frutta = ['mela', 'pera', 'banana', 'kiwi'];

// Aggiunta in coda e in testa
frutta.push('uva');           // ['mela', 'pera', 'banana', 'kiwi', 'uva']
frutta.unshift('arancia');    // ['arancia', 'mela', 'pera', 'banana', 'kiwi', 'uva']

// Rimozione in coda e in testa
frutta.pop();                 // rimuove 'uva'
frutta.shift();               // rimuove 'arancia'

// Ricerca
console.log(frutta.includes('pera'));    // true
console.log(frutta.indexOf('banana'));    // 2

// Lettura per indice con .at()
console.log(frutta.at(1));    // 'pera'

// Copia parziale senza modificare l'originale
const primi = frutta.slice(0, 2); // ['mela', 'pera']

// Rimozione e sostituzione con .splice()
frutta.splice(0, 1);             // rimuove 'mela'
frutta.splice(1, 1, 'fragola', 'pesca'); // sostituisce un elemento con due

// Inversione dell'ordine
frutta.reverse();

// Sovrascrittura di tutti gli elementi
frutta.fill('limone');

// Unione in stringa
const lista = frutta.join(', ');
```

CAPITOLO 4

Destructuring

NOTA

Il destructuring permette di estrarre uno o più valori da un array e assegnarli a variabili separate in una sola riga.

Funziona seguendo l'ordine degli elementi: il primo nome nella struttura di destructuring riceve il primo valore dell'array, il secondo il secondo, e così via.

Senza destructuring, per estrarre i valori da un array devi scrivere una riga per ogni elemento, usando l'indice. Con il destructuring lo fai in una sola istruzione, dichiarando i nomi delle variabili tra parentesi quadre a sinistra dell'uguale. Il risultato è lo stesso, ma il codice è più diretto e più semplice da leggere.

Se non ti interessa un elemento puoi saltarlo lasciando una virgola vuota al suo posto. Se invece vuoi raccogliere tutti i valori rimanenti in un nuovo array puoi usare `...rest` come ultimo elemento della struttura di destructuring: tutto ciò che non è stato estratto finisce lì dentro.

Esempio

```
const coordinate = ['Roma', 41.9, 12.4];

// Senza destructuring
const citta1 = coordinate[0];
const lat1   = coordinate[1];
const lon1   = coordinate[2];

// Con destructuring: stessa cosa in una riga
const [citta, lat, lon] = coordinate;

// Salto un elemento lasciando la virgola vuota
const [nome, , longitudine] = coordinate;

// Raccolgo i valori rimanenti in un nuovo array con ...rest
const stagioni = ['primavera', 'estate', 'autunno', 'inverno'];
const [prima, seconda, ...ultime] = stagioni;

console.log(prima);    // 'primavera'
console.log(ultime);   // ['autunno', 'inverno']
```

CAPITOLO 5

Spread operator

NOTA

Lo spread operator si scrive `...` davanti al nome di un array e ne espande tutti gli elementi nel punto in cui viene usato.

È utile per clonare un array, unirne più di uno o aggiungere valori senza modificare l'originale.

Quando assegni un array a una nuova variabile con il semplice uguale (`=`), non stai creando una copia: stai puntando allo stesso array. Qualsiasi modifica sulla nuova variabile cambia anche l'originale. Lo spread risolve questo problema: `[...array]` crea una copia indipendente, così le due variabili non si influenzano più a vicenda.

Lo spread è utile anche per unire più array in uno solo o per inserire nuovi valori in posizioni precise senza modificare le liste originali. Si scrive semplicemente mettendo `...` davanti a ciascun array all'interno delle parentesi quadre del nuovo array.

Esempio

```
const originale = ['uno', 'due', 'tre'];

// Assegnazione diretta: non è una copia, puntano allo stesso array
const nonCopia = originale;
nonCopia.push('quattro');
console.log(originale); // ['uno', 'due', 'tre', 'quattro'] - L'originale è cambiato!

// Clonazione con spread: copia indipendente
const copia = [...originale];
copia.push('cinque');
console.log(originale); // non cambia
console.log(copia);     // ['uno', 'due', 'tre', 'quattro', 'cinque']

// Unione di due array
const primiTre = ['a', 'b', 'c'];
const secondiTre = ['d', 'e', 'f'];
const tutti = [...primiTre, ...secondiTre]; // ['a', 'b', 'c', 'd', 'e', 'f']

// Aggiunta di valori in mezzo senza toccare gli originali
const conExtra = [...primiTre, 'X', 'Y', ...secondiTre];
```